

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №3
по дисциплине «Системы параллельной обработки данных»
Тема: Группы процессов и коммутаторы

Студент гр. 1306

Шаврин А.П.

Преподаватель

Татаринов Ю.С.

Санкт-Петербург

2025

Цель работы.

Изучить, как использовать коммуниторы библиотеке MPI. Реализовать программу, использующую функцию `MPI_Comm_split` для создания коммуниторов групп процессов как основную часть для решения задачи. Проанализировать работу программы.

Постановка задачи.

Вариант 10. В каждом процессе дано целое число N , которое может принимать два значения: 1 и 2 (имеется хотя бы один процесс с каждым из возможных значений). Кроме того, в каждом процессе дано целое число A . Используя функцию `MPI_Comm_split` и одну коллективную операцию пересылки данных, переслать числа A , данные в процессах с $N = 1$, во все процессы с $N = 1$, а числа A , данные в процессах с $N = 2$, во все процессы с $N = 2$. Во всех процессах вывести полученные числа в порядке возрастания рангов переславших их процессов (включая число, полученное из этого же процесса). Указание. Программа должна содержать единственный вызов функции `MPI_Comm_split`, создающий оба требуемых коммунитора (каждый в соответствующей группе процессов).

Выполнение работы.

Реализация началась с инициализации MPI в функции `main`: вызов `MPI_Init` для запуска среды, получение ранга (`MPI_Comm_rank`) и общего числа процессов (`MPI_Comm_size`).

Основная логика программы была вынесена в функцию `processTask` для структурирования кода и упрощения отладки. В этой функции каждый процесс определяет свои значения N и A согласно условию задачи:

- N принимает значения 1 или 2 в зависимости от четности ранга процесса (четные - $N=1$, нечетные - $N=2$) для обеспечения наличия хотя бы одного процесса со значениями 1 и 2.
- A вычисляется как `ранг_процесса * 10` для демонстрации уникальности данных в каждом процессе.

Далее следует ключевая операция работы - вызов `MPI_Comm_split`. Эта функция создает новые коммуникаторы, разделяя все процессы на группы по значению параметра `N`. Процессы с одинаковым значением `N` попадают в одну группу, при этом порядок рангов внутри группы определяется параметром `key`. В данном случае `key` = исходный ранг процесса, таким образом гарантируется, что вывод будет в порядке возрастания исходных рангов процессов.

После создания группового коммуникатора каждый процесс получает информацию о своем ранге и размере группы в новом коммуникаторе с помощью `MPI_Comm_rank` и `MPI_Comm_size`.

Основная коллективная операция программы - `MPI_Allgather` - выполняется внутри каждой созданной группы. Эта операция собирает значения `A` со всех процессов группы и рассылает собранный массив обратно всем процессам группы. Таким образом, каждый процесс получает все значения `A` из своей группы.

Завершается функция выводом полученных значений в порядке возрастания рангов процессов в группе, после чего созданный коммуникатор освобождается с помощью `MPI_Comm_free`.

Для визуального представления алгоритма нарисована сеть Петри для 4х процессов представленная на рисунке 1. Начальными для каждого процесса являются состояния готовности к генерации, конечными являются состояния, когда каждый процесс получил все числа `A` от остальных процессов в группе и вывел их в консоль.

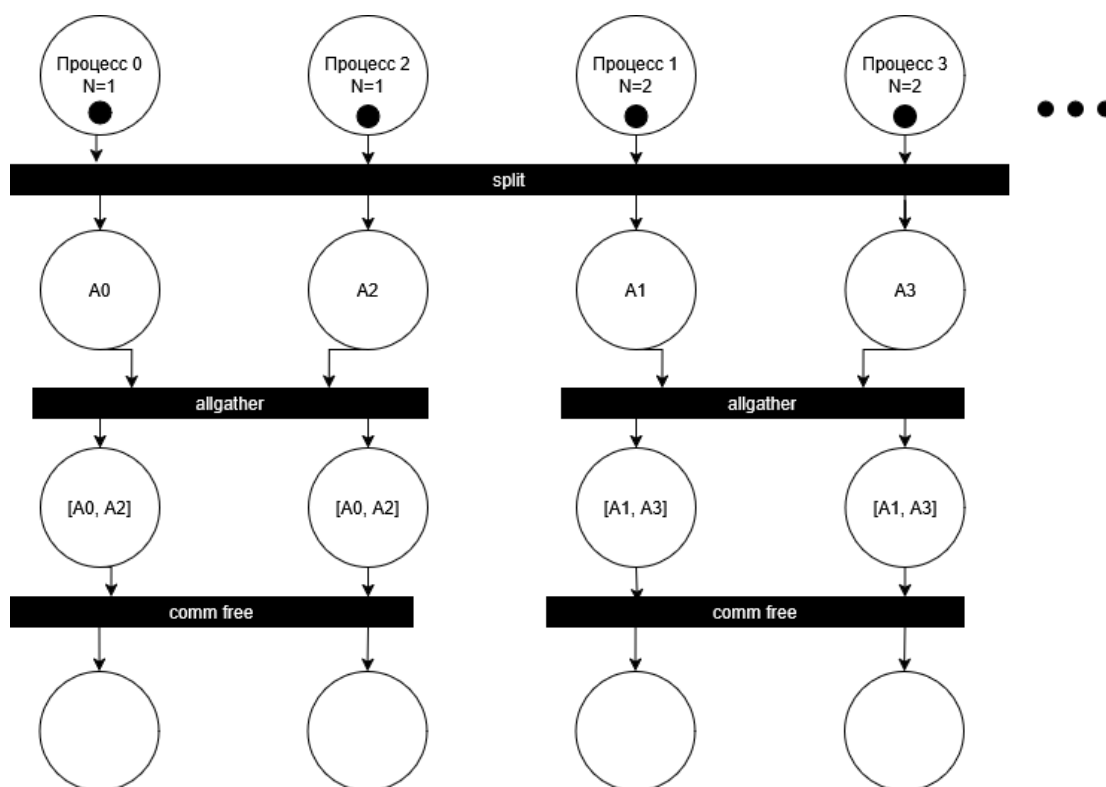


Рисунок 1 – Сеть Петри для 4х процессов.

Исследуем время выполнения программы при различном размере задачи для различного числа процессов. Результаты экспериментов представлены в таблице 1. Для каждого числа процессов значения приведены в усреднение на 50 запусков программы (время работы программы считалось по среднему времени завершения всех процессов).

Таблица 1.

Сложность задачи	Число процессов										
	2	3	4	5	6	8	10	12	16	24	32
0	0.44	0.71	1.9	4.22	26.13	4.02	10.28	10.79	21.65	44.3	70.62

По данным из таблицы можно сделать несколько ключевых выводов о поведении программы.

Во-первых, как можно видеть на рисунке 2, время выполнения программы зависит от числа MPI-процессов нелинейно. При увеличении количества процессов от 2 до 5 наблюдается плавное увеличение времени выполнения за счет увеличения нагрузки на ядра. Однако начиная с 6 процессов происходит резкий рост времени. Это связано с тем, что на

рассматриваемом узле доступно 6 физических ядер, и при запуске 6 MPI-процессов вычислительные ядра оказываются полностью заняты. В этом случае системным службам, а также внутренним потокам, обслуживающим коммуникации MPI, не остается свободных ресурсов. В результате операции коллективного обмена, в частности MPI_Allgather, выполняются значительно медленнее, поскольку все процессы вынуждены ждать освобождения процессорного времени для обработки передаваемых данных.

После превышения порога в 6 процессов время снова уменьшается, но не возвращается к минимальным значениям. Это связано с тем, что при большем числе процессов планировщик операционной системы начинает распределять их с переключением контекста, и процессы приходят к точке синхронизации MPI_Allgather не одновременно. Возникает эффект рассинхронизации: процессы не формируют плотного барьера, и внутренние коммуникации MPI получают возможность выполняться между переключениями задач. В результате пик задержек, связанный с одновременным вызовом коллективных операций всеми процессами, сглаживается, и время выполнения уменьшается. Однако общее время всё равно возрастает с ростом числа процессов из-за накладных расходов на контекстные переключения и усложнение коммуникационных шаблонов.

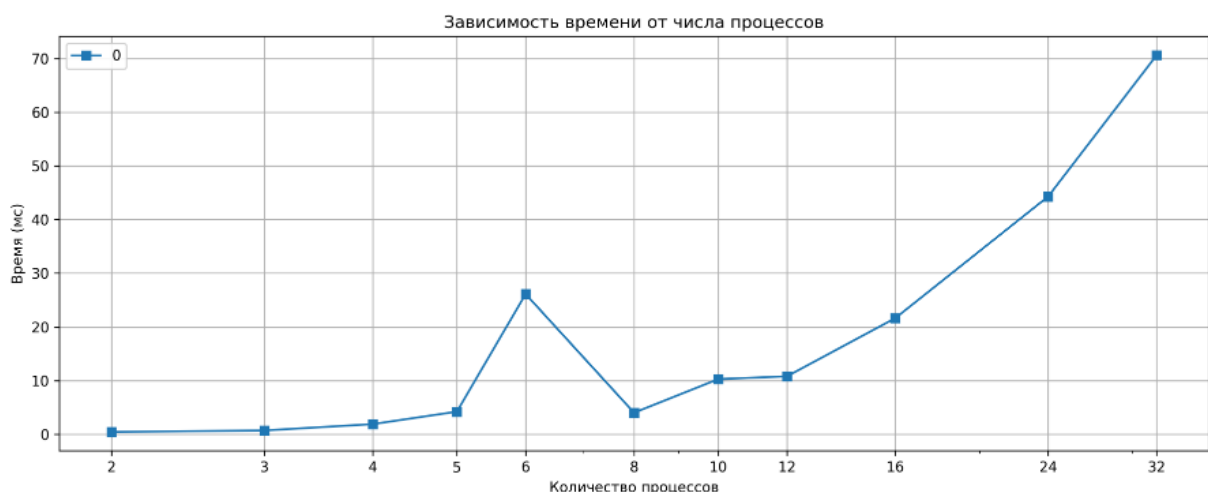


Рисунок 2. Зависимость времени от числа процессов.

Рассчитаем ускорение программы по формуле:

$$S_p(n) = \frac{T_1(n)}{T_p(n)}, \text{ где } p - \text{число процессов, а } n - \text{размеры поля}$$

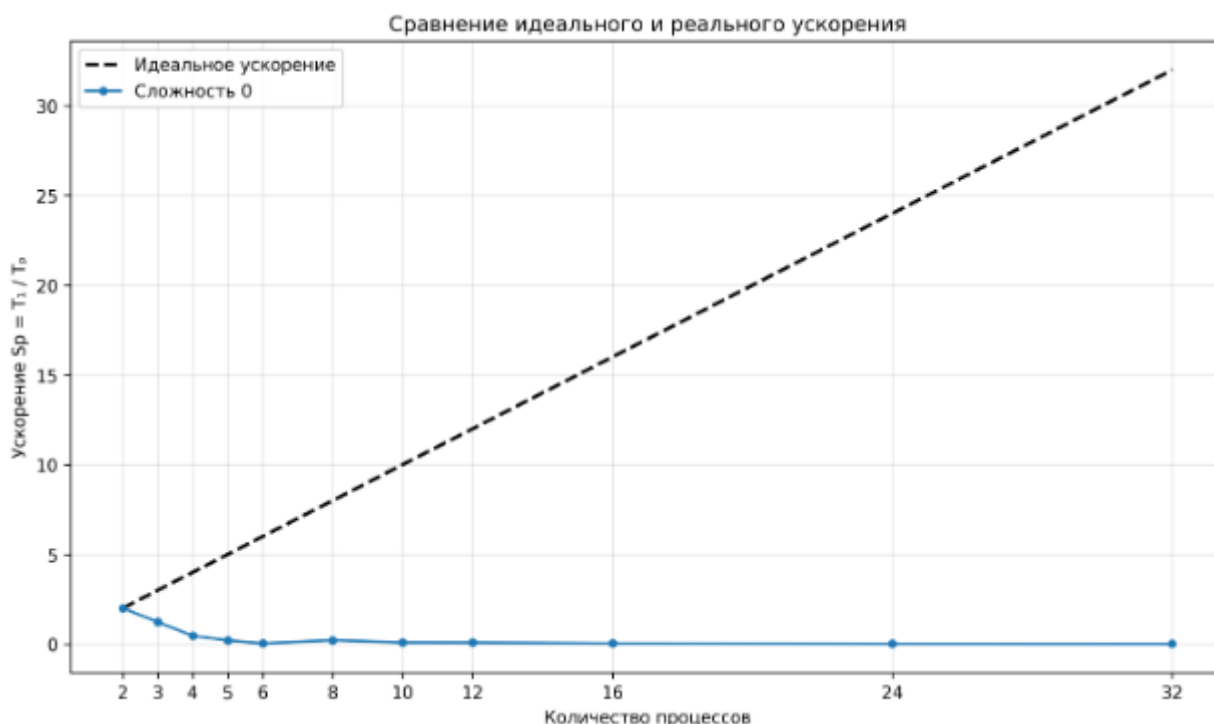


Рисунок 3. Ускорение программы для разной сложности задачи при увеличении числа процессов.

Как видно из рисунка 3, данные демонстрируют отсутствие реального ускорения при увеличении числа процессов. Каждый процесс независимо выполняет свой цикл. Однако эффект от этого распараллеливания полностью теряется из-за операции `MPI_Allgather`, которая работает как синхронизационный барьер. Видно, что ускорение падает до 6 процессов, а потом почти равно нулю. Это связано с тем, что на машине, на которой проводились тесты, всего 6 ядер. При превышении данного числа процессы делят ядра через быстрое переключение контекста, когда планировщик ОС поочередно выделяет им кванты времени на одних и тех же физических ядрах. Соответственно время выполнения начинает сильно расти из-за накладных расходов, а ускорение близится к 0.

Исходный код программы находится в приложении А, результат выполнения программы для 7 процессов находится в приложении Б.

Выводы.

В ходе лабораторной работы была разработана программа на C++ с использованием MPI, демонстрирующая разбиение набора процессов на подгруппы с помощью функции `MPI_Comm_split` и выполнение коллективного обмена данными внутри каждой из таких групп. В зависимости от значения параметра `N` процессы были разделены на два коммуникатора, после чего числовые значения `A` были переданы в пределах соответствующих групп с использованием единственной коллективной операции `MPI_Allgather`. Полученные данные корректно выводились каждым процессом в порядке возрастания рангов внутри новой группы, что подтверждает правильность формирования коммуникаторов и синхронизации обмена.

Анализ производительности показал, что время выполнения программы растёт нелинейно с увеличением числа процессов. Наибольший скачок наблюдается при числе процессов, равном количеству физических ядер (в данном случае 6), поскольку в этот момент все вычислительные ресурсы оказываются полностью заняты, и системным сервисам, а также внутренним механизмам MPI, не остаётся свободного процессорного времени. Это приводит к увеличению задержек в операции `MPI_Allgather`, которая является синхронизационным барьером. При дальнейшем увеличении числа процессов время частично уменьшается за счёт рассинхронизации их выполнения, однако при этом продолжают расти накладные расходы на переключение контекста. В результате программа не демонстрирует ускорения, так как вычислительная нагрузка не распределяется, а взаимодействие между процессами определяется затратными коллективными операциями.

Таким образом, работа подтвердила, что эффективность параллельных программ на MPI существенно зависит от характера межпроцессорного обмена и от соответствия числа процессов аппаратным ресурсам. Использование коммуникаторов позволяет гибко группировать процессы, однако для достижения масштабируемости необходимо минимизировать

синхронизирующие коллективные операции либо переработать структуру вычислений таким образом, чтобы сократить взаимные ожидания процессов.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
main.cpp
#include <mpi.h>
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <fstream>

void processTask(int world_rank, int world_size) {
    int N, A;

    if (world_rank % 2 == 0) {
        N = 1;
    } else {
        N = 2;
    }

    A = world_rank * 10;
    std::cout << "Process " << world_rank << " (A = " << A << ", N = " <<
N << ")" << std::endl;

    MPI_Comm_group_comm;
    MPI_Comm_split(MPI_COMM_WORLD, N, world_rank, &group_comm);

    int group_rank, group_size;
    MPI_Comm_rank(group_comm, &group_rank);
    MPI_Comm_size(group_comm, &group_size);

    std::vector<int> gathered_A(group_size);
    MPI_Allgather(&A, 1, MPI_INT,
                 gathered_A.data(), 1, MPI_INT,
                 group_comm);

    std::cout << "Process " << world_rank << " (N=" << N << "): ";
    std::cout << "Received values: ";
    for (int i = 0; i < group_size; i++) {
        std::cout << gathered_A[i];
        if (i < group_size - 1) std::cout << " ";
    }
    std::cout << std::endl;

    MPI_Comm_free(&group_comm);
}

void getTimeStatistic(int rank, int size, double processTime) {
    std::vector<double> allTimes;
    if (rank == 0) {
        allTimes.resize(size);
    }
    MPI_Gather(&processTime, 1, MPI_DOUBLE,
              allTimes.data(), 1, MPI_DOUBLE,
              0, MPI_COMM_WORLD);
}
```

```

        if (rank == 0) {
            double maxTime = *std::max_element(allTimes.begin(),
allTimes.end());
            double avgTime = std::accumulate(allTimes.begin(),
allTimes.end(), 0.0) / size;

            std::cout << "\n=== РЕЗУЛЬТАТЫ ДЛЯ " << size << " ПРОЦЕССОВ ==="
<< std::endl;
            std::cout << "Максимальное время: " << maxTime << " мс" <<
std::endl;
            std::cout << "Среднее время: " << avgTime << " мс" << std::endl;

            std::ofstream file("results.txt", std::ios::app);
            if (file.is_open()) {
                file << size << " " << maxTime << " " << avgTime <<
std::endl;
                file.close();
                std::cout << "Результаты записаны в results.txt" <<
std::endl;
            } else {
                std::cout << "Ошибка открытия файла!" << std::endl;
            }
        }
    }

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int world_rank, world_size;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    double startTime = MPI_Wtime();
    processTask(world_rank, world_size);
    double endTime = MPI_Wtime();
    double processTime = (endTime - startTime) * 1000;

    getTimeStatistic(world_rank, world_size, processTime);

    MPI_Finalize();
    return 0;
}

```

ПРИЛОЖЕНИЕ Б

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

```
Process 2 (A = 20, N = 1)
Process 3 (A = 30, N = 2)
Process 4 (A = 40, N = 1)
Process 5 (A = 50, N = 2)
Process 1 (A = 10, N = 2)
Process 0 (A = 0, N = 1)
Process 6 (A = 60, N = 1)
Process 4 (N=1): Received values: 0 20 40 60
Process 0 (N=1): Received values: 0 20 40 60
Process 6 (N=1): Received values: 0 20 40 60
Process 5 (N=2): Received values: 10 30 50
Process 2 (N=1): Received values: 0 20 40 60
Process 3 (N=2): Received values: 10 30 50
Process 1 (N=2): Received values: 10 30 50
```